

二维数组

基本使用

```
int arr[10] = {1,2,3,4,5,6,7}; // 一维数组

{1,2,3,4,5,6,7}

{1,2,3,4,5,6,7}

{1,2,3,4,5,6,7} // 多个一维数组，组成二维数组。
```

定义语法

```
1 int arr[行][列] = {数组元素}
2 int arr[2][3] =
3 {
4     {2, 5, 8},           // 第0行
5     {7, 9, 10}           // 第1行
6 };
7 // 常规写法:
8 int arr[3][5] = {{2, 3, 54, 56, 7}, {2, 67, 4, 35, 9}, {1, 4, 9, 3, 78}};
```

打印

```
1 // 自动补齐的 for 自带的 size_t 来源:
2 查看: 方法1: 右键 — 转到定义
3       方法2: F12
4 typedef unsigned int size_t; // 给 unsigned int 起别名, 叫 size_t
5 -----
6 // 以下是打印 2 维数组的方法:
7 int arr[3][5] = { {2, 3, 54, 56, 7}, {2, 67, 4, 35, 9}, {1, 4, 9, 3, 78} };
8
9 for (size_t i = 0; i < 3; i++) // 行
10 {
11     for (size_t j = 0; j < 5; j++) // 列
12     {
13         printf("%d ", arr[i][j]);
14     }
15     printf("\n");
16 }
```

特性

- 数组大小

```
1 printf("数组大小: %u\n", sizeof(arr));
```

- 一行大小

```
1 | printf("数组一行大小: %u\n", sizeof(arr[0]));
```

- 一个元素大小

```
1 | printf("数组一个元素大小: %u\n", sizeof(arr[0][0]));
```

- 行数

```
1 | int row = sizeof(arr) / sizeof(arr[0]); // 数组总大小 / 每行大小
```

- 列数

```
1 | int col = sizeof(arr[0]) / sizeof(arr[0][0]); // 一行大小 / 每个元素大小
```

- 地址合一

```
1 | 数组的地址 == 数组的首元素地址 == 数组的首行地址
2 | printf("%p\n", arr); // 数组的首地址
3 | printf("%p\n", arr[0]); // 数组首行地址
4 | printf("%p\n", &arr[0][0]); // 数组首元素的地址
```

初始化

常规初始化

```
1 | int arr[3][5] = { {2, 3, 54, 56, 7}, {2, 67, 4, 35, 9}, {1, 4, 9, 3, 78} };
```

不完全初始化

```
1 | int arr[3][5] = {{2,3}, {2, 67, 4}, {1, 4, 16, 78}}; // 未被初始化的数值, 为0
2 | int arr[3][5] = {0}; // 初值全部为0的二维数组
3 | int arr[3][5] = { 2, 3, 4, 5, 6, 7, 8, 9, 99, 2, 16, 78}; // 【少见】系统自动分配行列
```

不完全指定行列初始化

```
1 | int arr[][] = {1, 23, 4, 56, 7, 8}; 【错误】 // 二维数组定义, 至少需要指定 列值。
2 |
3 | int arr[][2] = { 1, 23, 4, 56, 7, 8, 10}; // 可以不指定行值。
4 |
5 | int row = sizeof(arr) / sizeof(arr[0]);
6 | int col = sizeof(arr[0]) / sizeof(arr[0][0]);
7 |
8 | for (size_t i = 0; i < row; i++) // 行
9 | {
10 |     for (size_t j = 0; j < col; j++) // 列
11 |     {
12 |         printf("%d ", arr[i][j]);
13 |     }
```

```
14     printf("\n");
15 }
```

练习

- 求出5名学生3门功课的总成绩。（总成绩：一个学生的总成绩。一门功课的总成绩）

```
1  int main(void)
2  {
3      int scores[5][3];    // 5个学生， 3门功课
4
5      int row = sizeof(scores) / sizeof(scores[0]);
6      int col = sizeof(scores[0]) / sizeof(scores[0][0]);
7
8      // 获取 5 个学生 3门功课成绩
9      for (size_t i = 0; i < row; i++)
10     {
11         for (size_t j = 0; j < col; j++)
12         {
13             scanf("%d", &scores[i][j]);
14         }
15     }
16     // 一门功课的总成绩
17     for (size_t i = 0; i < col; i++)    // 一次取出，每个学生的 一门功课
18     {
19         int sum = 0;    // 累加每门功课的分数。
20         for (size_t j = 0; j < row; j++)    // 每门功课第几个学生
21         {
22             sum += scores[j][i];
23         }
24         printf("第%d门功课总成绩: %d\n", i + 1, sum);
25     }
26
27     // 求每个学生的总成绩
28     for (size_t i = 0; i < row; i++)    // 每个学生
29     {
30         int sum = 0;    // 累加每个学生的成绩。
31
32         for (size_t j = 0; j < col; j++)    // 每个学生的成绩
33         {
34             sum += scores[i][j];    // sum = sum + scores[i][j];
35         }
36         printf("第%d个学生的总成绩为: %d\n", i+1, sum);
37     }
38     //printf("-----\n");
39
40     ///// 打印 5 个学生 3门功课成绩
41     //for (size_t i = 0; i < row; i++)
42     //{
43     //    for (size_t j = 0; j < col; j++)
44     //    {
45     //        printf("%d ", scores[i][j]);
46     //    }
47     //    printf("\n");
48 }
```

```

48     //}
49     system("pause");
50     return EXIT_SUCCESS;
51 }

```

多维数组(了解)

- 三维数组: [层][行][列]
- 语法: 类型名 数组名[层][行][列]

```

1  int arr[3][3][4] =
2  {
3      {
4          {{12, 3, 4, 5}}, // 第0行
5          {{12, 3, 4, 5}}, // 第1行
6          {{12, 3, 4, 5}}  // 第2行
7      }, // 第0层
8      {
9          {}, // 第0行
10         {}, // 第1行
11         {}  // 第2行
12     }    // 第1层
13 };

```

- 打印:

```

1  int main(void)
2  {
3      int arr[3][4][2] =
4      {
5          {
6              {1, 2},
7              {3, 4},
8              {5, 6},
9              {7, 8}
10         },
11         {
12             {12, 24},
13             {31, 49},
14             {5, 46},
15             {17, 88}
16         },
17         {
18             {122, 24},
19             {311, 419},
20             {15, 46},
21             {17, 188}
22         }
23     };
24
25     for (size_t i = 0; i < 3; i++) // 层
26     {
27         for (size_t j = 0; j < 4; j++) // 行

```

```

28     {
29         for (size_t k = 0; k < 2; k++) // 列
30         {
31             printf("%d ", arr[i][j][k]);
32         }
33         printf("\n");
34     }
35     printf("\n\n");
36 }
37
38 system("pause");
39 return EXIT_SUCCESS;
40 }
41

```

int arr[2][3][4] = {1,2,3,4,5,6,7,8,9};

int arr[][3][4] = {1,2,3,4,5,6,7,8,9}; // 层数, 可以省略。

数组首地址 == 首层地址 == 首层首行地址 == 首元素地址。

4维、5维、6维、。。。。N维

int arr[2][3]

short arr[2][3]

float arr[2]

long long arr[2][3][5]

字符串

- 一串字符。C语言中, 一定使用 '\0' 结束。

字符数组和字符串的区别

- 字符数组

```

1 char str1[5] = {'h', 'e', 'l', 'l', 'o'}; // 不是字符串, 没有 \0

```

- 字符串

```

1 char str2[6] = {'h', 'e', 'l', 'l', 'o', '\0'}; 【麻烦】
2
3 char str3[6] = "hello"; // 自动带有 \0 结束标志。

```

字符串输出

- printf(" %s ")
 - 打印字符串, 挨着从字符串的第一个字符顺序向后打印, 打印到 '\0' 结束。没碰到 '\0' 不结束。

- 'a' != "a" ('a', \0)
- 'abc' 是一个错误定义！既不是字符串，也不是有效字符。

其他格式匹配符

- %Ns:
 - 显示 N个字符的字符串，不足N用空格向右填充。

```
1 printf("|%9s|\n", str);
```

- %0Ns:
 - 显示 N个字符的字符串，不足N用0向左填充。

```
1 printf("|%09s|\n", str);
```

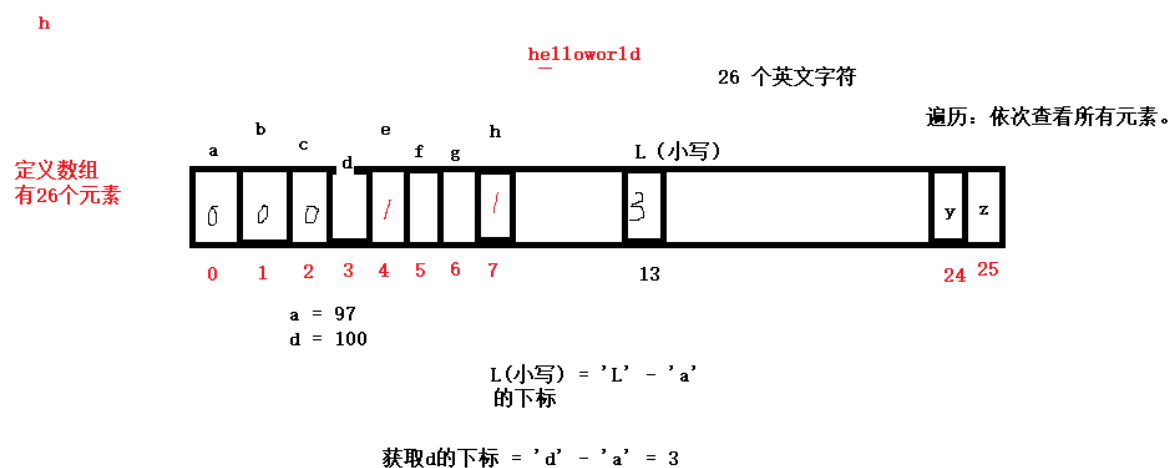
- %-Ns:
 - 显示 N个字符的字符串，不足N用空格向左填充。

```
1 | printf("|%-9s|\n", str);
```

- **%%**: 与字符串无直接关系。
 - 显示一个%, 转义字符 '\', 对%无效。转义%, 使用%本身。
 - 输出【10 % 3 = 1】: `printf("10 %% 3 = 1\n");`

练习：

- 键盘输入字符串，存至str[]中，统计每个字母出现的次数。
- 分析



```
1 int main(void)
2 {
3     char str[1024] = { 0 };    // 定义有10个元素的字符数组，初值均为0
4
5     for (size_t i = 0; i < 11; i++)
```

```

6      {
7          scanf("%c", &str[i]);    // helloworld
8      }
9      // 定义一个有26个元素的数组，初始化成 0
10     char count[26] = { 0 };    // 代码26个英文字符出现的次数。
11
12     for (size_t i = 0; i < 11; i++)
13     {
14         int index = str[i] - 'a';    // 提取每个字符，在 count 表中对应的下标。
15         count[index]++;    // 向对应字符，位置++，代表该字符出现了一次。
16     }
17
18     // 循环遍历 count 数组，打印出每个字符，出现的次数。
19     for (size_t i = 0; i < 26; i++)
20     {
21         if (count[i] != 0)    // 0 == \0
22         {
23             printf("%c字符，在字符串%s中，出现了%d次\n", i+'a', str, count[i]);
24         }
25     }
26
27     system("pause");
28     return EXIT_SUCCESS;
29 }

```

scanf 获取字符串

```

1 char str[1024] = {0};    // 定义字符串存储的空间，保证足够大。
2 scanf("%s", str);

```

- 注意事项：
 - 用于存储字符串的空间，必须足够大！防止溢出。
 - %s 遇到 空格 和 \n 终止。

```

1 char str[1024] = {0};
2 scanf("%s", str);    --- 获取 “hello world haha xixi” 字符串
3 printf("%s", str);    ---- 输出 hello

```

- 借助“正则表达式”，可以获取带有空格的字符串。scanf("%[^\n]", str);

字符串操作函数

gets

- 从(键盘)标准输入 stdin 获取字符串。返回字符串首地址，可以获取带有空格的字符串，不保存 \n，将其替换为 \0

```

1  #include <stdio.h>
2
3  char *gets(char *s);    // char * 等价于 char []
4      参数：用来存储字符串的空间地址。
5      返回值：返回实际获取到的字符串的首地址。
6  // 示例
7  char str[1024] = {0};
8  printf("获取的字符串为: %s\n", gets(str));

```

fgets

- 从 (键盘) 标准输入 stdin 获取字符串。一定会给字符串预留\0空间。可以获取带有空格的字符串。如果空间足够，保留\n，如果空间不足，不保留\n。

```

1  #include <stdio.h>
2  char *fgets(char *s, int size, FILE *stream);    // char * 等价于 char []
3      参1：用来存储字符串的空间地址。
4      参2：空间的大小。（严格对应实际空间的大小）
5      参3：读取字符串的位置。 -- stdin（键盘）
6
7      返回值：返回实际获取到的字符串的首地址。
8
9  // 示例：
10 char str[15];
11 int len = sizeof(str);
12 printf("获取到的字符串为: %s\n", fgets(str, len, stdin));

```

puts

- 将字符串输出到 屏幕 标准输出 stdout。输出后会自动向屏幕输出 \n

```

1  // printf("%s\n", "hello")
2
3  #include <stdio.h>
4
5  int puts(const char *s);    // char * 等价于 char []
6      参：待 写入到屏幕的字符串。
7      返回值：
8          成功：0， 失败：-1
9  // 示例：
10 char str[] = "hello world";
11 int ret = puts(str);
12 printf("ret = %d\n", ret);

```

fputs

- 将字符串输出到 屏幕 标准输出 stdout。不自动添加\n 字符。


```

1  #include <stdio.h>
2
3  int fputs(const char * str, FILE * stream); // char * 等价于 char []
4      参1: 待写出到屏幕的字符串。
5      参2: 写出的位置。—— stdout 标准输出。屏幕。
6      返回值:
7          成功: 0,  失败: -1
8  // 示例:
9      char str[] = "hello world\n";
10
11     int ret = fputs(str, stdout);
12
13     printf("ret = %d\n", ret);

```

strlen

- 作用: 获取一个字符串有效字符个数 (字符串的长度)。不含\0 (碰到\0 结束)

```

1  #include <string.h>
2
3  size_t strlen(const char *s);
4      参: 待 求长度的字符串
5      返回: 有效字符个数
6  // 示例:
7      char str[] = "hello world\n";
8      printf("有效长度=%u\n", strlen(str)); // 不含有 \0 长度
9      printf("sizeof=%u\n", sizeof(str)); // 含有 \0 长度

```

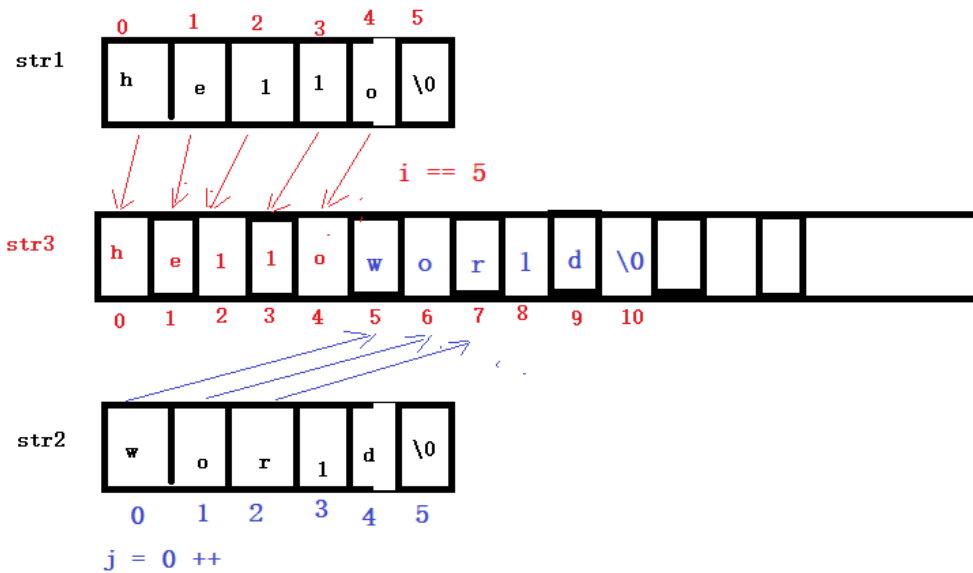
- 实现 strlen 函数

```

1  int main(void)
2  {
3      char str[] = "hello world";
4
5      int i = 0;
6      while (str[i] != '\0')
7      {
8          i++;
9      }
10     printf("不含\\0的字符串长度为: %d\n", i);
11
12     printf("strlen = %d\n", strlen(str)) ;
13
14     system("pause");
15     return EXIT_SUCCESS;
16 }

```

字符串追加



```

1  int main(void)
2  {
3      char str1[] = "hello";
4      char str2[] = "world";
5      char str3[100];
6
7      // 循环将 str1 中的字符，依次写入到 str3 中
8      int i = 0;
9      while (str1[i] != '\0')
10     {
11         str3[i] = str1[i];
12         i++;
13     }           // 循环结束，str3 = 【hello】无 \0
14     // printf("i = %d\n", i); --- 循环结束为 5 。
15
16     int j = 0;   // 循环 str2
17
18     // 循环将 str2 中的字符，接着str1的内容顺序写入到 str3 中
19     while (str2[j]) // while(str2[i] != 0) == while (str2[i] != '\0')
20     {
21         str3[i+j] = str2[j];
22         j++;
23     }           // 循环结束，str3 = 【helloworld】无 \0
24
25     // 手动添加 \0 结束标记
26     str3[i + j] = '\0';
27
28     printf("str3 = %s\n", str3);
29
30     system("pause");
31     return EXIT_SUCCESS;
32 }

```

函数的作用

- 提高代码复用率。
- 提高程序模块化组织性。

函数分类

- 系统库函数：标准C库。 libc
 1. 必须要引入头文件 `#include <xxx.h>` ---- 函数声明。
 2. 根据函数库函数原型，调用函数。
- 用户自定义函数：
 - `bubble_sort()`、`myPrint()`
 - 除了需要提供函数原型之外，还需要提供函数实现。

使用函数

- 函数定义、函数声明、函数调用

函数定义

- 函数定义必须包含“函数原型”和“函数体”
 - 函数原型：返回值类型 + 函数名 + 形参列表。
 - 形参列表：形式参数列表。一定包含：类型名、形参名

```
1 // 加法函数
2 int add(int a, int b)
```

- 函数体：一对 {} 包裹函数实现。

```
1 // 例子：
2 int add(int a, int b)
3 {
4     int ret = a + b;
5     return ret;
6 }
7 // int test(char ch, short b, int arr[], int m)
```

函数调用

- 包含：函数名(实参列表);
 - 实参（实际参数）：在调用时，传参必须严格按照形参填充。（参数个数、类型、顺序）
 - 实参 在调用时，没有 类型描述符。

```
1 // 例子
2 int m = 10;
3 int n = 20;
4 int ret = add(m, n);
```

函数声明

- 包含：函数原型(返回值类型 + 函数名 + 形参列表) + “;”

```
1 // 例子
2 int add(int a, int b);
```

- 要求，在函数调用之前，编译器，必须见过函数定义。否则需要函数声明。
- 如果，没有函数声明，编译默认做“隐式声明”。
 - 隐式声明：【不要依赖】
 - 编译认为所有的函数，返回值都是 int。
 - 可以根据函数调用，推测函数原型。
- #include <xxx.h> 内部，包含 函数声明。

exit 函数

多文件编程